HCI Notes: Task Analysis and Conceptualizing Interaction

HCI - Weeks 2 & 3, Fall 2025

Abstract

These notes provide a comprehensive overview of task analysis techniques and the process of conceptualizing interaction in Human-Computer Interaction (HCI). Part I covers structured methods for analyzing and documenting user tasks, including Question, Option, Comment (QOC) for design rationale, Hierarchical Task Analysis (HTA) for breaking down tasks, and State Transition Networks (STN) for modeling system behavior. Part II delves into the foundational stage of design: conceptualizing interaction. It explores the transition from a problem space to a design space, the role of assumptions and claims, the development of conceptual models, and the use of interface metaphors. It also details various interaction types and other forms of conceptual knowledge that guide design and research in HCI.

Contents

Part I
Task Analysis Techniques

1 Question, Option, Comment (QOC)

1.1 Introduction to QOC

Question, Option, Comment (QOC) is a semi-formal notation used to represent the design space and rationale behind design decisions. It provides a structured way to explore alternatives and document why certain choices were made. This method is crucial for teamwork, as it makes the design thinking process explicit and reviewable.

The Structure of QOC

- **Question:** Poses a key design problem or issue that needs to be addressed. Questions highlight the major design challenges.
- **Option:** Proposes one or more potential solutions (answers) to the question. Options represent the different design choices available.
- **Comment (or Criteria/Argument):** States the arguments for and against each option. Comments capture the trade-offs, pros, and cons, providing the rationale for the final decision.

1.2 QOC Examples

The following examples illustrate how QOC can be applied to common design problems.

1.2.1 Example 1: User Login System (Slide 2)

- **Question:** How should users log in to the system?
- **Options:**
  1. Username + Password
  2. Biometric (fingerprint/face scan)
  3. Single Sign-On (Google/Facebook/LinkedIn)
- **Comments:**
  - *For Username + Password:* Passwords are a familiar method for users, but they are prone to being forgotten and can be vulnerable to hacking.
  - *For Biometrics:* Biometrics are highly convenient and reduce friction, but they raise significant privacy concerns and require specialized hardware (e.g., fingerprint scanner, 3D camera), which may not be available on all devices.
  - *For Single Sign-On (SSO):* SSO greatly reduces friction for the user by leveraging existing accounts. However, it creates a dependency on third-party services, which could be a point of failure or raise data-sharing concerns.

1.2.2 Example 2: Navigation in a Mobile App (Slide 3)

- **Question:** How should users navigate between the main sections of the app?
- **Options:**
  1. Bottom navigation bar
  2. Hamburger menu
  3. Swipe gestures

- **Comments:**
  - *For Bottom navigation bar:* This pattern is highly visible and easy to use, making top-level navigation always accessible. However, it is limited to a small number of items (typically 4-5) to avoid clutter.
  - *For Hamburger menu:* Hides navigation options, which reduces clutter on the screen. The major drawback is that it harms discoverability, as users cannot see the primary navigation options at a glance.
  - *For Swipe gestures:* Gestures can feel natural and modern, but they suffer from significant learnability issues (users have to be taught or discover them) and can pose accessibility problems for some users.

1.2.3 Further Examples (Slides 4-6)

- **Error Notifications:** Pop-ups force attention but interrupt workflow, while inline errors provide context but may be overlooked.
- **Online Shopping Checkout:** One-click is fast but risks accidental purchases, while a multi-step process adds friction but provides clarity.
- **Feedback Collection:** Star ratings are quick but lack detail, text comments are rich but time-consuming, and emojis are engaging but can be vague.

2 Hierarchical Task Analysis (HTA)

2.1 Introduction to HTA

Hierarchical Task Analysis (HTA) is a structured method for analyzing and representing how work is performed. It involves breaking down a high-level task (or goal) into a hierarchy of sub-tasks and operations.

Core Concepts of HTA

- **Goal:** The overall objective the user wants to achieve.
- **Sub-tasks:** The constituent steps required to achieve the goal. These sub-tasks can be broken down further into more detailed steps.
- **Plan:** Describes the conditions under which the sub-tasks should be performed and in what sequence (e.g., "Do 1.1, then 1.2, then 1.3. When finished, do 2.0.").

HTA is an excellent tool for understanding the complexity of a task, identifying potential problems, and informing the design of an interface that effectively supports the user's workflow.

2.2 HTA Examples

2.2.1 Example 1: Online Shopping Checkout (Slide 8)

This diagram shows a classic HTA for the goal: "Buy a product online."

- **Top-Level Goal (0):** Buy a product online.
- **Level 1 Sub-tasks:**
  1. Search for product
  2. Add product to cart
  3. Proceed to checkout
  4. Receive Confirmation
- **Level 2 Sub-tasks (for Task 1):**

1.1 Enter keywords in search bar

1.2 Browse results

1.3 Select desired item

- **Level 2 Sub-tasks (for Task 3):**

3.1 Enter shipping details

3.2 Select payment method

3.3 Confirm Order

2.2.2 Example 2: Ordering Food from FoodPanda (Slide 10)

This is a highly detailed, linear representation of an HTA. It breaks down the entire process from launching the app to providing feedback into 11 main steps, each with its own sub-steps. This level of detail is useful for identifying every single user action and system interaction required to complete the overall goal. For example, Step 5.0 "Select food items" is broken down into "5.1 Browse menu categories," "5.2 Choose dish," "5.3 Customize," and "5.4 Add to cart."

3 State Transition Network (STN)

3.1 Introduction to STN

A **State Transition Network (STN)**, also known as a State Transition Diagram or Finite State Machine, is a model of behavior that describes how a system moves between different states in response to user actions or events.

Components of an STN

- **States:** A particular condition or status of the system (e.g., "Empty," "Full," "Logged In," "Logged Out"). Represented by circles or rounded rectangles.
- **Transitions:** A change from one state to another. Represented by arrows.
- **Actions/Events:** The trigger that causes a transition to occur (e.g., a user clicks a button, a sensor detects something). Represented by a label on an arrow.

STNs are useful for modeling the dialog structure of an interface and ensuring that all possible system states and user pathways are accounted for.

3.2 STN Examples

3.2.1 Example 1: Bottle in a Bottling Plant (Slide 13)

This diagram models the life cycle of a bottle.

- **States:** Empty, Full, Broken, Sealed.
- **Transitions & Actions:**
  - From *Empty*, the action 'squirt(n)' can lead to *Full* (if capacity is reached) or back to *Empty* (if not yet full).
  - From *Full*, the action 'cap' leads to the *Sealed* state.
  - The 'break' action can occur at any point, leading to the *Broken* state.

3.2.2 Example 2: A Chess Game (Slide 14)

This STN models the flow of a chess game.

- **States:** Start, White's turn, Black's turn, Black wins, White wins, Draw.
- **Transitions & Actions:**
  - The game alternates between "White's turn" and "Black's turn" based on the 'white moves' and 'black moves' actions.
  - From either turn state, an action of 'checkmate' or 'stalemate' can occur, leading to a final game state (Win, Loss, or Draw).

Part II
Conceptualizing Interaction

4 From Problem Space to Design Space

4.1 Introduction to Conceptualizing Design

The initial phase of any design project involves **conceptualization**—forming an idea or a concept of what the product will do, who it is for, and how it will support users. This is a critical step that moves the design team from a vague idea to a concrete proof of concept.

Why Conceptualize?

- **To Scrutinize Ideas:** It forces the team to examine vague ideas and assumptions about the product's benefits and user experience.
- **To Assess Feasibility:** It prompts key questions like, "How realistic is this to develop?"
- **To Determine Value:** It helps to answer, "How desirable and useful will this be for users?"

4.1.1 Design Example: The Robot Waiter (Slides 2-6)

To understand this process, consider two different concepts for a "robot waiter":

1. **The Conversational Waiter:** A humanoid robot designed to take orders and entertain customers through conversation.
2. **The Functional Server:** A "tool-like" robot on wheels designed purely to transport food from the kitchen to tables, saving human waiters' time.

The second concept is generally preferable because it addresses a clear, real-world problem (improving waiter efficiency) rather than relying on the risky assumption that customers will enjoy having conversations with a robot.

4.2 Assumptions and Claims

A key part of conceptualization is identifying and questioning the underlying assumptions and claims of a design idea.

- **Assumption:** Taking something for granted that needs further investigation.
  - *Example Assumption:* "The robot could entertain customers by having a conversation with them." (Is this actually desirable?)
- **Claim:** Stating something to be true when it is still open to question.

– *Example Claim:* "It will raise revenue by attracting more people to the restaurant." (Is this a sustainable business model or just a novelty?)

By working through assumptions, asking critical questions, and articulating concerns, the design team moves from the **problem space** (understanding the problem) to the **design space** (exploring potential solutions).

5 Conceptual Models

5.1 What is a Conceptual Model?

A conceptual model is a high-level description of how a system is organized and operates. It is not the user interface itself but the underlying concept and logic.

Definition

A conceptual model is "...a high-level description of how a system is organized and operates" (Johnson and Henderson, 2002). It provides a working strategy and a framework of general concepts and their interrelations.

The best conceptual models are often those that seem obvious and simple, making the operations they support feel intuitive to use.

5.2 Components of a Conceptual Model

A conceptual model is typically composed of:

1. **Metaphors and Analogies:** These convey how to use the product by relating it to something familiar.
2. **Concepts:** The core objects, attributes, and operations the user is exposed to (e.g., saving a file, organizing photos into albums).
3. **Relationships:** The mappings between these concepts (e.g., how a shopping cart relates to a checkout process).

5.3 Interface Metaphors

Interface metaphors are a powerful tool for making unfamiliar systems understandable.

- **Definition:** An interface designed to be similar to a physical entity or a known activity, which helps users understand the "unfamiliar" by relating it to the "familiar."
- **Classic Example: The Desktop Metaphor.** The computer's graphical interface is conceptualized as a desktop, with files, folders, and a trash/recycle bin. This helps users understand digital file management by relating it to organizing papers on a physical desk.
- **The Card Metaphor:** A very popular metaphor used in apps and websites (e.g., Google Now, fast food kiosks). Content is structured into "cards," which are familiar, can be easily flicked through or sorted, and break information into meaningful chunks.

5.3.1 Problems with Metaphors

While useful, metaphors can have drawbacks. They can:

- **Break Rules:** The recycle bin is on the desktop, not under it, breaking the physical metaphor's logic.
- **Constrain Design:** Over-reliance on a metaphor can limit a designer's imagination.
- **Not Scale Well:** A simple metaphor may fail to describe a highly complex system.

6 Interaction Types

An interaction type is a way of conceptualizing what the user is doing when interacting with a system.

6.1 The Five Main Interaction Types

1. **Instructing:** The user issues commands to the system. This is common for repetitive tasks (e.g., telling a word processor to "save" or "print"). It is quick and efficient.
2. **Conversing:** The user interacts with the system as if having a conversation. This ranges from simple menu-driven voice systems to complex natural language chatbots (e.g., Siri, Google Assistant). It is familiar but can lead to misunderstandings if the system cannot parse the user's input.
3. **Manipulating:** The user interacts with virtual objects by manipulating them directly (e.g., dragging, selecting, zooming). Ben Shneiderman's theory of **Direct Manipulation** outlines its core properties:
   - Continuous representation of objects.
   - Physical actions instead of complex syntax.
   - Rapid, reversible actions with immediate feedback.

   Direct manipulation is easy to learn and gives users a sense of control, but it is not suitable for all tasks (e.g., spell-checking is better delegated as an instruction).
4. **Exploring:** The user moves through a virtual or physical environment. This is central to 3D software, games, and immersive VR/AR experiences where users can explore data or virtual objects in a spatial context.
5. **Responding:** The system initiates the interaction by alerting the user to something it "thinks" is of interest. For example, a fitness tracker notifying you that you've reached a goal, or a map app alerting you to nearby friends. This is automatic and proactive but can become tiresome if the notifications are irrelevant or too frequent.

6.2 Choosing an Interaction Type

The right interaction type depends on the task:

- **Direct Manipulation** is good for "doing" tasks (drawing, driving).
- **Instructing** is good for repetitive tasks (file management).
- **Conversing** is good for information retrieval (finding movie times).
- **Hybrid models** are often used to support multiple ways of performing the same action.

7 Other Forms of Conceptual Knowledge

Beyond conceptual models and interaction types, design and research are guided by other high-level concepts.

- **Paradigms:** A general approach or set of shared assumptions adopted by a research community (e.g., the move from the desktop paradigm to ubiquitous computing).
- **Visions:** A driving force that frames research by imagining life in the future (e.g., Apple's 1987 "Knowledge Navigator" video, visions of smart cities).
- **Theories:** An explanation of a phenomenon that can be used to predict what users will do (e.g., information processing theory from cognitive psychology).
- **Models:** A simplification of an HCI phenomenon used to predict and evaluate designs (e.g., Don Norman's model of the Seven Stages of Action).
- **Frameworks:** A set of interrelated concepts or questions to guide analysis and design.

7.1 A Classic HCI Framework: Don Norman's Model

Don Norman's framework describes the relationship between the designer, the system, and the user:

1. **The Designer's Model:** The designer's mental model of how the system works.
2. **The System Image:** How the system is actually presented to the user through its interface, documentation, etc. This is the only way the user can learn about the system.
3. **The User's Model:** The user's mental model of how the system works, which is formed through their interaction with the system image.

The goal of a designer is to ensure the System Image accurately and clearly conveys the Designer's Model, so that the User's Model will match.

8 Summary

- Developing a conceptual model is a crucial first step in design. It involves understanding the problem space, clarifying assumptions, and specifying how the design will support user activities.
- A conceptual model is a high-level description of a product, defining what users can do with it and what concepts they need to understand to interact with it.
- Interaction types (Instructing, Conversing, Manipulating, Exploring, Responding) provide a way of thinking about how to support user's activities.
- Paradigms, visions, theories, models, and frameworks are all forms of conceptual knowledge that help frame and guide design and research in HCI.